UNIVERSIDAD DE LAS AMÉRICAS PUEBLA

PRÁCTICA 3

Sistemas embebidos con FPGA

Contador ascendente–descendente de 9 bits con signo
Intervalo de −256 a 255 en la DE10-Lite

Integrantes:	Roberto Orozco Ramírez ID: 184595
	Adam Del Pozo Ontiveros ID: 185611
	Enrique Bernal Martín Prieto ID: 184878
Carrera:	Ingeniería en Sistemas Computacionales
Materia:	Sistemas Embebidos
Grupo:	[completar]
Profesor:	[completar]
Fecha:	[completar]

Resumen

En esta práctica se diseñó, simuló e implementó un contador ascendente–descendente de 9 bits con signo en Verilog. El valor recorre el intervalo completo de complemento a dos, de −256 a 255, con control de reinicio, pausa, conteo ascendente y descendente, y un selector de actualización de 1 s o 100 ms. El valor se muestra en decimal en HEX2–HEX0, y HEX3 indica el signo. El diseño usa un único reloj de 50 MHz y pulsos de habilitación (`tick`) de un ciclo.

1. Objetivo

Diseñar e implementar en Verilog un contador ascendente–descendente de 9 bits con signo, verificar su comportamiento mediante simulación funcional y demostrarlo en la tarjeta DE10-Lite, incluyendo los límites del intervalo y el cambio de signo.

2. Fundamento teórico

2.1. Representación con signo en 9 bits

Un número de 9 bits en complemento a dos representa x con $-2^8 \leq x \leq 2^8 - 1$, es decir, de -256 a 255. El bit más significativo `count[8]` funciona como bit de signo: vale 0 para no negativos y 1 para negativos. Como hay $2^9 = 512$ patrones, el rango es simétrico salvo por un patrón extra en el lado negativo: $-256 = 1_0000_0000$, que no tiene contraparte positiva.

Como la aritmética es modular, el contador da la vuelta sin saturar:

$$255 + 1 \longrightarrow -256, \qquad -256 - 1 \longrightarrow 255.$$

2.2. Unidad de control y ruta de datos

La unidad de control es una máquina de estados con tres estados: **PAUSA** (conserva el valor), **ASCENDENTE** (suma 1 en cada `tick`) y **DESCENDENTE** (resta 1 en cada `tick`). La ruta de datos es un registro con signo de 9 bits que solo cambia cuando llega un `tick`.

2.3. Pulso de habilitación

La DE10-Lite entrega un reloj de 50 MHz. Para un intervalo T el número de ciclos es $N = f_{\mathrm{CLK}}\,T$:

$$N_{1\,\mathrm{s}} = 50\,000\,000, \qquad N_{100\,\mathrm{ms}} = 5\,000\,000.$$

El divisor cuenta de 0 a $N-1$ y genera un pulso de un solo ciclo. No se crea ningún reloj nuevo, así que todo el diseño sigue en el dominio del reloj principal.

2.4. Acondicionamiento de botones

Los pulsadores mecánicos rebotan y son asíncronos respecto al reloj. El diseño usa un sincronizador de dos flip-flops, un filtro antirrebote de 10 ms (500 000 ciclos) y un pulso de un ciclo por pulsación válida.

3. Material y software

- Intel Quartus Prime (proyecto para el dispositivo 10M50DAF484C7G de la DE10-Lite).
- Tarjeta FPGA DE10-Lite, cable USB Blaster y manual de la tarjeta.
- Icarus Verilog 12.0 para simulación funcional.

4. Diseño implementado

4.1. Arquitectura y máquina de estados

- `tick_generator`: genera `tick` de un ciclo cada 1 s o cada 100 ms según `speed_sel`. Se usa `>=` en la comparación para que un cambio del selector no deje al divisor esperando un valor ya rebasado. En pausa el divisor se mantiene en cero.
- `signed_counter_fsm`: FSM de tres estados y contador con signo. La prioridad es RESET > PAUSA > INCREMENTO > DECREMENTO. El contador solo se actualiza en `tick`.
- `signed_display_9bit`: convierte el valor a magnitud decimal (centenas, decenas y unidades) y a 7 segmentos activos en bajo. HEX3 enciende solo el segmento g cuando el número es negativo. Los ceros a la izquierda se apagan.
- `button_onepulse`: sincronizador de dos FF, antirrebote de 10 ms y pulso de un ciclo.
- `counter_9bit_top`: módulo superior que integra todos los bloques.

Asignación de recursos usada en la tarjeta: KEY0 = reset, KEY1 = pausa, pulsador GPIO = incremento, pulsador GPIO = decremento, SW0 = selector de frecuencia (0: 1 s, 1: 100 ms). Los números de pin se toman del archivo de asignaciones de la DE10-Lite del laboratorio.

4.2. Tabla de LEDs

LED	Señal	Significado
LEDR[0]	PAUSA	Estado de pausa
LEDR[1]	ASCENDENTE	Conteo hacia arriba
LEDR[2]	DESCENDENTE	Conteo hacia abajo
LEDR[3]	`speed_sync`	Actualización cada 100 ms
LEDR[9]	`count[8]`	Número negativo

5. Simulación funcional

Se simularon dos testbenches con Icarus Verilog. El primero (`tb_counter_fsm.v`) sigue la tabla de pruebas de la práctica. El segundo (`tb_display_tick.v`) verifica el decodificador de 7 segmentos y el periodo del tick. Ambos reportan `PASS`/`FAIL` automáticamente.

5.1. Resultados de la tabla de pruebas

Valor actual	Evento	Valor obtenido	Estado
0	RESET	0	PAUSA
0	INC y después tick	1	ASCENDENTE
58	tick	59	ASCENDENTE
255	tick	−256	ASCENDENTE
0	DEC y después tick	−1	DESCENDENTE
−58	tick	−59	DESCENDENTE
−256	tick	255	DESCENDENTE
45	PAUSA y varios ticks	45	PAUSA
−123	RESET	0	PAUSA

La salida completa de la simulación del FSM dice: *TODAS LAS PRUEBAS PASARON* (16 de 16 verificaciones). Además se probó que PAUSA tiene prioridad sobre INCREMENTO cuando llegan en el mismo ciclo.

5.2. Decodificador y periodo del tick

Valor	HEX3	HEX2	HEX1	HEX0
255	(apagado)	2	5	5
−256	-	2	5	6
−1	-	(apagado)	(apagado)	1
0	(apagado)	(apagado)	(apagado)	0
45	(apagado)	(apagado)	4	5
−123	-	1	2	3
100	(apagado)	1	0	0

El periodo medido entre dos ticks a 100 ms fue de **5 000 000 ciclos**, igual al valor esperado.

5.3. Formas de onda

[**Insertar aquí capturas de la simulación de los tres estados de control** (ASCENDENTE, DESCENDENTE y PAUSA) exportadas desde la vista de ondas del simulador o de Quartus.]

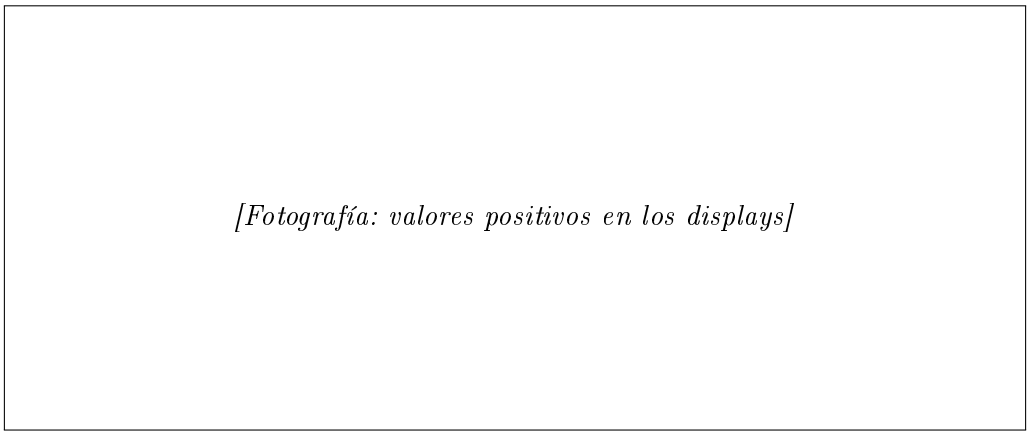
[Captura: transiciones 255 → −256 y −256 → 255]

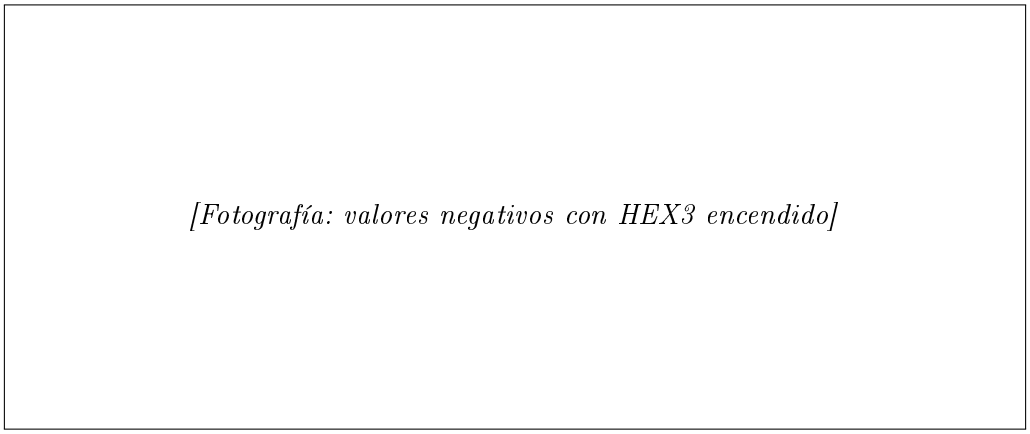
6. Demostración en la DE10-Lite

6.1. Compilación en Quartus Prime

[**Insertar aquí el reporte de utilización de recursos** generado por Quartus Prime: elementos lógicos, registros, pines y memoria. Este paso se hace en Quartus, no en el simulador.]

6.2. Evidencia en la tarjeta





6.3. Comparación experimental: 1 s contra 100 ms

Selector	Cambio observado por segundo (cuenta)	Valor a los 10 s	Observaciones
SW0 = 0 (1 s)	[completar]	[completar]	[completar]
SW0 = 1 (100 ms)	[completar]	[completar]	[completar]

6.4. Pruebas en hardware

- Verificación de transiciones $255 \rightarrow -256$ y $-256 \rightarrow 255$: [completar: resultado observado].
- Pausa conserva el valor y reinicio lleva a cero: [completar].
- Controles en ambas frecuencias: [completar].

7. Conclusiones

[Completar con la conclusión del equipo. Puntos sugeridos: la representación en complemento a dos explica el rango asimétrico y el wrap-around; separar unidad de control y ruta de datos simplifica la verificación; el pulso `tick` mantiene un único dominio de reloj y evita relojes derivados con lógica combinacional, que generan sesgo y problemas de temporización.]

8. Cuestionario

1. **¿Por qué 9 bits con signo representan de -256 a 255 y no de -255 a 255?** Con 9 bits hay $2^9 = 512$ patrones. Se reparten 256 negativos (-256 a -1), 0 y 255 positivos. El patrón 1_0000_0000 representa -256, que no tiene contraparte positiva, así que el rango no es simétrico.

2. **¿Qué patrón binario representa -256?** 1_0000_0000 (bit de signo en 1 y el resto en 0).

3. **¿Qué ocurre al incrementar 255 y al decrementar -256?** La aritmética es modular: $255 + 1$ da -256 y $-256 - 1$ da 255. No hay saturación; el registro simplemente da la vuelta.

4. **¿Qué diferencia hay entre generar un reloj nuevo y generar un pulso de habilitación?** Un reloj derivado de lógica combinacional tiene retrasos distintos a los del reloj principal, lo que causa sesgo de reloj y violaciones de temporización, y crea dominios de reloj que hay que sincronizar. Un pulso de habilitación de un ciclo mantiene todo el diseño en el dominio de 50 MHz: los registros siguen cambiando en el mismo flanco y solo se actualizan cuando el pulso está activo.

5. **¿Por qué deben sincronizarse las entradas de los botones?** Son asíncronas respecto al reloj de la FPGA. Si cambian justo cerca de un flanco, el flip-flop puede entrar en metaestabilidad. Un sincronizador de dos flip-flops reduce ese riesgo.

6. **¿Qué problema evita el circuito antirrebote?** Un pulsador mecánico produce varios cambios rápidos en una sola pulsación. Sin filtro, el contador contaría varias pulsaciones donde solo hubo una. El filtro de 10 ms acepta el cambio solo cuando la señal permanece estable.

7. **¿Cuántos ciclos de 50 MHz corresponden a 100 ms?** $0{,}1\,\mathrm{s} \times 50 \times 10^6\,\mathrm{Hz} = 5\,000\,000$ ciclos.

8. **¿Qué recursos adicionales pueden requerir los operadores de división y módulo durante la síntesis?** Como el divisor y el módulo son por constantes, se sintetizan como lógica de multiplicación por constantes y sumadores, lo que consume elementos lógicos adicionales. En el decodificador de displays, este costo es pequeño; un algoritmo *double dabble* puede reducirlo si el área es crítica.

9. Bibliografía

- M. M. Mano y M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6.a ed., Pearson, 2018.
- S. Brown y Z. Vranesic, *Fundamentals of Digital Logic with Verilog Design*, 3.a ed., McGraw-Hill, 2014.
- Terasic, *DE10-Lite User Manual.*
- Intel, *Quartus Prime Design Software Documentation.*

A. Código fuente

A.1. Módulo superior

```
// Modulo superior: contador ascendente-descendente de 9 bits con signo, DE10-Lite.
// Botones activos en bajo. SW0 selecciona la velocidad (0: 1 s, 1: 100 ms).
module counter_9bit_top (
    input  wire       CLOCK_50,
    input  wire       BTN_RESET_N,  // KEY0
    input  wire       BTN_PAUSE_N,  // KEY1
    input  wire       BTN_INC_N,    // GPIO
    input  wire       BTN_DEC_N,    // GPIO
    input  wire       SW_FREQ,      // SW0
    output wire [6:0] HEX0,
    output wire [6:0] HEX1,
    output wire [6:0] HEX2,
    output wire [6:0] HEX3,
    output wire [9:0] LEDR
);

    wire reset = ~BTN_RESET_N;

    wire        pause_pulse, inc_pulse, dec_pulse, tick;
    wire [1:0]  state;
    wire        running = (state != 2'd0);
    wire signed [8:0] count;

    // Sincronizador de dos FF para el selector de frecuencia
    reg speed_meta, speed_sync;
    always @(posedge CLOCK_50 or posedge reset) begin
        if (reset) begin
            speed_meta <= 1'b0;
            speed_sync <= 1'b0;
        end else begin
            speed_meta <= SW_FREQ;
            speed_sync <= speed_meta;
        end
    end

    button_onepulse u_pause (.clk(CLOCK_50), .reset(reset), .button_n(BTN_PAUSE_N),
        .press_pulse(pause_pulse));
    button_onepulse u_inc   (.clk(CLOCK_50), .reset(reset), .button_n(BTN_INC_N),
        .press_pulse(inc_pulse));
    button_onepulse u_dec   (.clk(CLOCK_50), .reset(reset), .button_n(BTN_DEC_N),
        .press_pulse(dec_pulse));

    tick_generator u_tick (
        .clk(CLOCK_50), .reset(reset), .enable(running),
        .speed_sel(speed_sync), .tick(tick)
    );

    signed_counter_fsm u_counter (
        .clk(CLOCK_50), .reset(reset), .tick(tick),
        .pause_pulse(pause_pulse), .inc_pulse(inc_pulse), .dec_pulse(dec_pulse),
        .count(count), .state(state)
    );

    signed_display_9bit u_display (
        .value(count), .HEX0(HEX0), .HEX1(HEX1), .HEX2(HEX2), .HEX3(HEX3)
    );

    // LEDR[0]: pausa, LEDR[1]: ascendente, LEDR[2]: descendente
    // LEDR[3]: 100 ms, LEDR[9]: numero negativo
    assign LEDR = {count[8], 5'b00000, speed_sync,
                   (state == 2'd2), (state == 2'd1), (state == 2'd0)};
```

```
59
60 endmodule
```

A.2. Unidad de control y ruta de datos

```
1  // Unidad de control (FSM) + ruta de datos del contador con signo de 9 bits.
2  // Rango: -256 a 255 en complemento a dos. El desbordamiento es modular:
3  //   255 + 1 -> -256   y   -256 - 1 -> 255
4  module signed_counter_fsm (
5      input  wire               clk,
6      input  wire               reset,        // activo en alto, maxima prioridad
7      input  wire               tick,         // pulso de habilitacion de 1 ciclo
8      input  wire               pause_pulse,  // prioridad 1
9      input  wire               inc_pulse,    // prioridad 2
10     input  wire               dec_pulse,    // prioridad 3
11     output reg  signed [8:0]  count,
12     output reg         [1:0]  state
13 );
14
15     localparam [1:0] S_PAUSE = 2'd0;
16     localparam [1:0] S_UP    = 2'd1;
17     localparam [1:0] S_DOWN  = 2'd2;
18
19     // Control: la prioridad es PAUSA > INCREMENTO > DECREMENTO
20     always @(posedge clk) begin
21         if (reset) begin
22             state <= S_PAUSE;
23             count <= 9'sd0;
24         end else begin
25             if (pause_pulse)
26                 state <= S_PAUSE;
27             else if (inc_pulse)
28                 state <= S_UP;
29             else if (dec_pulse)
30                 state <= S_DOWN;
31
32             // Ruta de datos: solo cambia con tick y segun el estado actual
33             if (tick) begin
34                 case (state)
35                     S_UP:    count <= count + 9'sd1;   // 255 -> -256 (wrap)
36                     S_DOWN:  count <= count - 9'sd1;   // -256 -> 255 (wrap)
37                     default: count <= count;           // PAUSA: conserva el valor
38                 endcase
39             end
40         end
41     end
42
43 endmodule
```

A.3. Decodificador de displays

```
1  // Muestra un numero con signo de 9 bits en decimal en HEX2..HEX0.
2  // HEX3 muestra el signo negativo (segmento g). Salidas activas en bajo (DE10-Lite)
       .
3  // Los ceros a la izquierda se apagan.
4  module signed_display_9bit (
5      input  wire signed [8:0] value,
6      output wire        [6:0] HEX0,
7      output wire        [6:0] HEX1,
8      output wire        [6:0] HEX2,
9      output wire        [6:0] HEX3
10 );
```

```
11
12      localparam [6:0] BLANK = 7'b111_1111;
13      localparam [6:0] MINUS = 7'b011_1111;  // solo g encendido
14
15      // Magnitud: 10 bits para representar |-256| = 256
16      wire signed [9:0] v10 = value;
17      wire        [9:0] mag = v10[9] ? (~v10 + 10'd1) : v10[9:0];
18
19      wire [3:0] units    = mag % 10;
20      wire [3:0] tens     = (mag / 10) % 10;
21      wire [3:0] hundreds = mag / 100;
22
23      wire hide_hundreds = (hundreds == 0);
24      wire hide_tens     = hide_hundreds && (tens == 0);
25
26      function [6:0] seg7;   // decodificador BCD -> 7 segmentos, activo en bajo
27          input [3:0] d;
28          begin
29              case (d)
30                  4'd0: seg7 = 7'b100_0000;
31                  4'd1: seg7 = 7'b111_1001;
32                  4'd2: seg7 = 7'b010_0100;
33                  4'd3: seg7 = 7'b011_0000;
34                  4'd4: seg7 = 7'b001_1001;
35                  4'd5: seg7 = 7'b001_0010;
36                  4'd6: seg7 = 7'b000_0010;
37                  4'd7: seg7 = 7'b111_1000;
38                  4'd8: seg7 = 7'b000_0000;
39                  4'd9: seg7 = 7'b001_0000;
40                  default: seg7 = 7'b111_1111;
41              endcase
42          end
43      endfunction
44
45      assign HEX0 = seg7(units);
46      assign HEX1 = hide_tens     ? BLANK : seg7(tens);
47      assign HEX2 = hide_hundreds ? BLANK : seg7(hundreds);
48      assign HEX3 = value[8]      ? MINUS : BLANK;
49
50  endmodule
```

A.4. Generador de tick

```
1   // Generador seleccionable de pulsos de habilitacion de un ciclo.
2   // speed_sel = 0 -> tick cada 1 s  (50 000 000 ciclos)
3   // speed_sel = 1 -> tick cada 100 ms (5 000 000 ciclos)
4   module tick_generator (
5       input  wire clk,
6       input  wire reset,
7       input  wire enable,
8       input  wire speed_sel,
9       output reg  tick
10  );
11
12      reg  [25:0] counter;
13      wire [25:0] terminal_count = speed_sel ? 26'd4_999_999 : 26'd49_999_999;
14
15      always @(posedge clk or posedge reset) begin
16          if (reset) begin
17              counter <= 26'd0;
18              tick    <= 1'b0;
19          end else if (!enable) begin
20              // En pausa el divisor se mantiene en cero
```

```
            counter <= 26'd0;
            tick    <= 1'b0;
        end else if (counter >= terminal_count) begin
            // >= evita quedarse esperando un valor ya rebasado al cambiar
                speed_sel
            counter <= 26'd0;
            tick    <= 1'b1;
        end else begin
            counter <= counter + 26'd1;
            tick    <= 1'b0;
        end
    end

endmodule
```

A.5. Antirrebote

```
// Acondicionador de boton activo en bajo: sincronizador de 2 FF,
// antirrebote de ~10 ms y pulso de un ciclo por pulsacion valida.
module button_onepulse (
    input  wire clk,
    input  wire reset,
    input  wire button_n,
    output reg  press_pulse
);

    // 500 000 ciclos / 50 MHz = 10 ms
    localparam [18:0] DEBOUNCE_COUNT = 19'd499_999;

    reg [1:0]  sync_ff;
    reg        stable_n;
    reg [18:0] counter;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            sync_ff     <= 2'b11;
            stable_n    <= 1'b1;
            counter     <= 19'd0;
            press_pulse <= 1'b0;
        end else begin
            sync_ff     <= {sync_ff[0], button_n};
            press_pulse <= 1'b0;

            if (sync_ff[1] == stable_n) begin
                counter <= 19'd0;
            end else if (counter == DEBOUNCE_COUNT) begin
                counter  <= 19'd0;
                stable_n <= sync_ff[1];
                // Transicion estable de 1 a 0: boton presionado
                if ((stable_n == 1'b1) && (sync_ff[1] == 1'b0))
                    press_pulse <= 1'b1;
            end else begin
                counter <= counter + 19'd1;
            end
        end
    end

endmodule
```